2026年4月23日

OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

OshiruZine-AI

2026年4月23日 号

目次

1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. GPT-5・Gemini 2.5・Claude 4——2026年「第四世代LLM」競争の技術的ブレークスルーを読み解く　*特集（約4000字）*
3. AIエージェント時代のガードレール設計——自律行動するAIをどう制御するか　*スタンダード（約2000字）*
4. ローカルLLM 2026年春——Llama 4とオンデバイス推論の現在地　*ショート（500〜1000字）*

Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

2026年に入り、AIコーディングアシスタントの勢力図は大きく塗り替わった。Anthropicの「Claude Code」とOpenAIの「Codex」は、もはや単なる補完ツールではなく、ソフトウェア開発のワークフローそのものを再定義する存在になっている。ターミナルに常駐するエージェントがコードベース全体を理解し、設計判断まで提案してくる——そんな世界がすでに日常だ。本コラムでは、これら2つのツールを最大限に活かすための最新ベストプラクティスを、実務の現場から整理してお届けする。

エージェント型開発の前提を理解する

Claude CodeもCodexも、2025年後半から2026年にかけて「エージェントモード」を大幅に強化してきた。Claude Codeはターミナルネイティブのエージェントとして、リポジトリのコンテキストを自律的に探索し、ファイルの作成・編集・テスト実行までを一気通貫で行う。一方、OpenAI Codexもクラウドサンドボックス上での自律実行機能を拡充し、複数タスクの並列処理が可能になった。

重要なのは、これらのツールが「チャットで質問に答える」段階を超え、「タスクを委任して結果を受け取る」段階に進化している点だ。開発者に求められるスキルセットも変わりつつある。正確なプロンプトを書く力に加えて、タスクの分解能力、成果物のレビュー能力、そしてエージェントが動作する環境を適切に設計する力が問われるようになった。

CLAUDE.mdとシステムプロンプトの設計が鍵

Claude Codeを使い込んでいる開発チームが口を揃えるのが、「CLAUDE.mdの品質がプロジェクトの生産性を決める」という事実だ。CLAUDE.mdはリポジトリのルートに配置する指示ファイルで、Claude Codeがタスクを実行する際のコンテキストとルールを定義する。

ベストプラクティスとしては、以下の要素を明記することが推奨される。プロジェクトのアーキテクチャ概要、使用しているフレームワークとそのバージョン、コーディング規約、テスト戦略、そして「やってはいけないこと」のリストだ。特に禁止事項の明示が効果的で、「既存のAPIインターフェースを変更しないこと」「環境変数をハードコードしないこと」といった制約を書いておくだけで、エージェントの暴走リスクは大幅に下がる。

Codex側でも同様に、カスタムインストラクションやリポジトリコンテキストの事前設定が重要だ。両ツールとも、与えるコンテキストの精度が出力の品質に直結するという原則は変わらない。

タスク分解と並列実行の戦略

大規模な機能開発をエージェントに任せる際、最もよくある失敗は「大きすぎるタスクを一度に渡す」ことだ。2026年現在、Claude Codeの拡張コンテキストウィンドウは大幅に広がったが、それでもタスクの粒度管理は生産性に直結する。

効果的なアプローチは、まずエージェントにタスクの分解計画を立てさせ、その計画をレビューしてから個別タスクを実行させるという2段階方式だ。Claude Codeではサブエージェントの起動が可能になっており、親エージェントが計画を立て、子エージェントが個別タスクを並列で実行するパターンが定着しつつある。Codexでも複数タスクの並列キュー投入が可能で、PR単位での自動生成が実用レベルに達している。

ただし、並列実行時には競合に注意が必要だ。同一ファイルへの同時編集を避けるため、タスク間の依存関係を明確にし、インターフェース境界を先に確定させてから実装に入るという手順が推奨される。

テスト駆動でエージェントを制御する

エージェント型開発において、テストコードの役割はこれまで以上に重要になっている。テストは単なる品質保証ではなく、エージェントに対する「仕様書」として機能するからだ。

実践的なワークフローとしては、まず人間がテストケースを書き（あるいはエージェントに書かせてレビューし）、そのテストを通す実装をエージェントに委任する。Claude Codeはテスト実行結果を自動的にフィードバックループに組み込み、失敗したテストを修正するまで反復する機能を備えている。この「テストファースト＋エージェント実装」の組み合わせが、現時点で最も安定した成果物を生み出すパターンだ。

Codexでもサンドボックス内でのテスト自動実行が標準化され、テストが通らないコードはPRとして提出されない仕組みが構築できる。CI/CDパイプラインとエージェントの統合は、もはやオプションではなく必須要件だ。

セキュリティとガバナンスの新常識

エージェントがコードを自律的に書く時代、セキュリティとガバナンスの考え方もアップデートが必要だ。Claude Codeには権限制御の仕組みが用意されており、ファイルシステムへのアクセス範囲やコマンド実行の許可レベルを細かく設定できる。本番環境への接続情報をエージェントのスコープから除外する、外部パッケージのインストールには人間の承認を必須にする、といった設定は初期段階で必ず行うべきだ。

また、エージェントが生成したコードのライセンス問題や、学習データに起因する脆弱性パターンの混入リスクにも目を配る必要がある。2026年初頭に複数の

セキュリティ研究チームが報告したように、AIが生成するコードには特定のパターンの脆弱性が統計的に偏在する傾向がある。SAST（静的アプリケーションセキュリティテスト）ツールをエージェントの出力パイプラインに組み込むことを強く推奨する。

人間の役割はどこにあるか

最後に、最も本質的な問いに触れておきたい。エージェントがこれほど高度になった今、開発者は何をすべきなのか。

答えは明確だ。アーキテクチャの意思決定、ビジネス要件の解釈、そしてエージェントの成果物に対する最終判断——これらは依然として人間の領域だ。Claude CodeもCodexも、驚くほど正確なコードを生成するが、「なぜそのコードが必要なのか」を理解しているわけではない。開発者の価値は、コードを書く速度ではなく、正しい問題を解く力にシフトしている。

エージェントを優秀なジュニアエンジニアとして扱い、明確な指示を出し、成果物を厳格にレビューし、全体のアーキテクチャに責任を持つ。この姿勢こそが、2026年のソフトウェア開発における最も重要なベストプラクティスだと言えるだろう。

GPT-5・Gemini 2.5・Claude 4——2026年「第四世代LLM」競争の技術的ブレークスルーを読み解く

2026年の春、私たちは大規模言語モデル（LLM）の地殻変動のただ中にいる。OpenAIのGPT-5が1月にロールアウトを開始し、GoogleのGemini 2.5が3月にフルスペックで公開、そしてAnthropicのClaude 4が4月上旬に登場した。わずか3か月の間に三大プレイヤーが揃って世代交代を果たした——この密度は、2023年の「GPT-4ショック」以来、見たことがない。

しかし、ベンチマークのリーダーボードを眺めるだけでは、この世代交代の本質は見えてこない。各社はそれぞれ異なる設計思想を研ぎ澄ませ、異なる課題を突破口に選んでいる。本稿では、アーキテクチャの進化、長コンテキスト処理、マルチモーダル性能、推論能力の4軸から三者を横断比較し、「結局どのモデルをどの場面で使うべきか」というエンジニアにとって最も切実な問いに実践的な指針を示したい。

「第四世代」とは何か——世代を分ける技術的断層線

そもそも「第四世代LLM」という呼称は業界でコンセンサスが取れたものではないが、本稿では便宜上、以下の技術的特徴を複数満たすモデルを第四世代と定義する。

1. **100万トークン超のネイティブコンテキスト**が実用的な精度で動作する
2. **ネイティブマルチモーダル**（テキスト・画像・音声・動画・コード実行）が単一モデル内で統合されている
3. **推論時計算スケーリング（inference-time compute scaling）** を明示的にサポートし、思考の「深さ」を動的に制御できる

4. **エージェント的振る舞い**——ツール呼び出し、計画立案、自己修正——がプロンプトエンジニアリングではなくアーキテクチャレベルで組み込まれている

第三世代（GPT-4o、Gemini 1.5 Pro、Claude 3.5 Sonnet）までは、これらのうち1つか2つを部分的に達成していた。第四世代では4つすべてが「当たり前」の基盤となった点が決定的に異なる。

アーキテクチャの進化——三者三様の哲学

GPT-5：Mixture-of-Experts の極致とダイナミックルーティング

GPT-5は、GPT-4で導入されたMixture-of-Experts（MoE）アーキテクチャをさらに推し進めた。公開されたテクニカルレポートによれば、推論時にアクティベートされるパラメータ数はモデル総パラメータの約8%に抑えられており、入力トークンの内容に応じて動的にエキスパートを組み合わせる「Adaptive Routing」機構が導入されている。これにより、数学的推論にはロジック特化エキスパート群が、創作文にはスタイル特化エキスパート群が重点的にアクティベートされる。

注目すべきは、推論コストの低減がモデル能力の向上と同時に実現されている点だ。GPT-4oと比較してAPI単価はトークンあたり約40%低下しており、これはMoEの効率化に加え、投機的デコーディング（speculative decoding）の洗練によるものと推測される。

Gemini 2.5：マルチモーダルネイティブの究極形

Gemini 2.5の設計思想は一貫して「マルチモーダルファースト」だ。前世代のGemini 1.5 Proが音声・動画のロングコンテキスト理解で業界を驚かせたが、2.5ではさらに踏み込み、テキスト・画像・音声・動画・3Dポイントクラウドの全モダリティが「同一の潜在空間」で処理される統合アーキテクチャを採用している。Google DeepMindの論文では、これを「Universal Latent Transformer」と呼んでいる。

実務上のインパクトが大きいのは、動画理解の精度だ。1時間超の動画を入力し、特定の場面の因果関係を問う質問に対して、従来モデルでは50%台だった精度が80%を超えるケースが報告されている。製造業の品質検査や医療画像の長時間モニタリングといったユースケースで、Geminiが唯一の実用的な選択肢になりつつある。

Claude 4：「信頼性」をアーキテクチャで実装する

Anthropicの設計哲学は、他の二者とは明確に異なる。Claude 4の最大の特徴は、推論プロセスの透明性と制御可能性に重点を置いた「Constitutional Architecture」とでも呼ぶべき内部構造にある。

具体的には、モデル内部に複数の「レビューレイヤー」が組み込まれており、生成途中のトークン列が安全性・事実整合性・論理一貫性のチェックを通過してからデコードされる多段階生成方式が採用されている。この仕組みにより、ハルシネーション率は前世代のClaude 3.5 Sonnet比で約60%低減したと報告されている。

また、Claude 4はモデルの推論過程を構造化された形式で出力する「Reasoning Trace」機能を標準搭載した。これは単なる思考連鎖（Chain-of-Thought）の表示ではなく、各推論ステップに対する信頼度スコアと、採用しなかった代替推論パスの要約が付与される。エンタープライズ用途——法務文書のレビュー、金融リスク分析、医療診断支援——でクリティカルな「説明可能性」の要件に対して、現時点で最も強力な回答を提示するモデルだ。

長コンテキスト処理——「入る」と「使える」は違う

2026年現在、100万トークン超のコンテキストウィンドウはもはやスペック表の数字としては珍しくない。GPT-5は256Kトークン（一部モードで1M）、Gemini 2.5は最大200万トークン、Claude 4は標準100万トークンをサポートする。しかしエンジニアが本当に気にすべきは、「情報がコンテキストに入るか」ではなく「コンテキスト内の情報を正確に検索・統合できるか」だ。

ここで興味深い差が出る。いわゆる「Needle-in-a-Haystack」テスト（長いコンテキストの中に埋め込まれた特定の情報を正しく取り出せるか）の発展版として、2026年初頭にMosaicMLが公開した「Multi-Needle Reasoning Benchmark（MNRB）」が業界標準になりつつある。MNRBでは、コンテキスト内の複数箇所に散在する情報を統合して初めて回答できる問題が出題される。

このベンチマークで、Gemini 2.5が200万トークンの全範囲でほぼ均一な精度を維持したのに対し、GPT-5は128Kトークンを超えるあたりから精度の緩やかな低下が見られ、Claude 4は100万トークン近辺でも精度を維持するが処理速度が顕著に低下する。つまり、**大量の非構造化データを丸ごと投入して横断分析する用途ではGemini 2.5が圧倒的**であり、**精度と速度のバランスが求められるリアルタイム用途ではGPT-5の実効的なスイートスポット（〜128K）が依然として優位**、そして**精度を最優先にするアーカイブ分析・法務レビューにはClaude 4の堅牢性が光る**という構図だ。

推論能力——「考える」と「正しく考える」の違い

推論能力の比較は、2026年の第四世代LLM競争において最もホットなトピックだ。きっかけは、2025年後半のo1/o3シリーズ以降に確立された「推論時計算スケーリング」パラダイムである。モデルが回答を出す前により多くの計算資源を使って「考える」ことで、数学・コード・論理パズルの正答率が劇的に向上する手法だ。

GPT-5はこの路線の正統進化形であり、推論予算を段階的に設定できる「Thinking Budget」パラメータをAPI経由で公開している。低予算設定（fast mode）ではGPT-4oクラスの速度と品質、高予算設定（deep mode）ではIMO（国際数学オリンピック）レベルの問題に対して従来のo3を超える正答率を達成する。

Gemini 2.5もGoogleのFlash Thinkingの系譜を引く推論モードを備えるが、特筆すべきは**マルチモーダル推論**との統合だ。グラフや図表が含まれる学術論文のPDFを入力し、図表とテキストの情報を統合して新しい仮説を提案する、とい

ったタスクで最も高いスコアを出す。テキストだけで完結しない「現実世界の推論」においてGeminiの優位性は明確だ。

Claude 4の推論アプローチは独特で、前述のReasoning Traceと組み合わさることで「推論の信頼度の自己較正（self-calibration）」が極めて正確だ。具体的には、Claude 4が「90%の確信度」と報告した回答が実際に正しい確率は88〜92%の範囲に収まるという検証結果が複数の研究グループから報告されている。これは他のモデルが往々にして過信（overconfidence）傾向を示すのとは対照的であり、**意思決定支援において「このモデルが自信を持っている回答は本当に信用できる**」という信頼の基盤を提供する。

エージェント性能——2026年の真の戦場

ベンチマークでは見えにくいが、2026年前半のLLM競争における最大の差別化要因は「エージェントとしての能力」だと筆者は考えている。SWE-bench Verifiedやτ-bench（ツール利用の自律エージェントベンチマーク）のスコアでは三者とも90%前後で拮抗しているが、実際のプロダクション環境でのエージェント運用では体感的な差がある。

GPT-5はOpenAIのエコシステム（Code Interpreter、Browsing、各種プラグイン）との統合が最も成熟しており、「すぐに動くエージェント」を構築する際の立ち上がりが最速だ。Claude 4はエージェントのループ中で自己修正する能力——いわゆる「リカバリー率」——が最も高く、長時間の自律タスクで暴走しにくい。Gemini 2.5はGoogle Cloudのインフラストラクチャとのネイティブ統合により、BigQuery、Vertex AI、Google Workspaceとの連携が圧倒的にスムーズで、エンタープライズのデータパイプラインに組み込む際のフリクションが最小だ。

実践的選定ガイド——あなたのユースケースに最適なモデルは？

以下は、筆者がこの3か月間、複数のプロジェクトで三モデルを併用した経験と、公開ベンチマーク・技術レポートを総合した選定指針だ。

コード生成・リファクタリング → GPT-5（Deep mode）がファーストチョイス。特に大規模コードベースの理解とリファクタリング提案の品質が高い。Claude 4はコードレビュー観点での安全性指摘に強み。

マルチモーダル分析（画像・動画・音声を含むタスク） → Gemini 2.5一択。特に動画理解と図表を含む文書分析は他のモデルとの差が顕著。

法務・コンプライアンス・医療など高信頼性が求められるドメイン → Claude 4のReasoning Traceと自己較正精度が極めて有用。説明可能性の要件がある場合は最有力候補。

大量文書の横断分析（RAGなしで処理したい場合） → Gemini 2.5の200万トークンコンテキストが最大の武器。ただし、精度を最優先するならClaude 4の100万トークンも十分候補になる。

リアルタイムチャット・カスタマーサポート → GPT-5のfast modeがレイテンシと品質のバランスで最も実用的。

自律エージェント・長時間タスク → Claude 4のリカバリー率と安定性を推奨。GPT-5はエコシステムの成熟度で選ぶケースも。

次に来る波——2026年後半に注目すべき動向

最後に、今後6か月で注目すべきトレンドを3つ挙げておきたい。

第一に、**オープンウェイトモデルの急追**だ。MetaのLlama 4（2026年2月公開）がすでにClaude 3.5 Sonnetクラスの性能をオープンウェイトで実現しており、年後半にはLlama 4の大型バリアントがプロプライエタリモデルとの差をさらに縮めるだろう。

第二に、**推論コストの劇的低下**。DistilledなモデルやSLM（Small Language Model）の進化により、第四世代の能力の80%を10分の1のコストで得られる時代が年内に来る可能性が高い。エッジデバイスでの動作も現実味を帯びてくる。

第三に、**規制の具体化**。EU AI Actの完全施行が2026年8月に迫っており、「推論過程の記録義務」「自動意思決定の説明可能性要件」が具体化する中で、Claude 4のようなアーキテクチャレベルでの透明性設計がコンプライアンス上の優位性に直結する局面が増えるだろう。

2026年のLLM競争は、もはや「どれが一番賢いか」という単純な問いでは語れない。各モデルは明確に異なる設計思想の下で異なる強みを研ぎ澄ませており、エンジニアに求められるのは「最強のモデルを選ぶ」ことではなく「課題に最適なモデルを見極める」リテラシーだ。本稿がその判断の一助となれば幸いである。

AIエージェント時代のガードレール設計——自律行動するAIをどう制御するか

「AIに任せたはずのタスクが、気づいたときには取り返しのつかない操作を完了していた」——これは仮定の話ではない。2025年後半から2026年にかけて、AIエージェントの商用導入が本格化するなかで、実際に複数の企業が"エージェント暴走インシデント"を経験し、その教訓がようやく業界全体のナレッジとして共有され始めている。ツール呼び出し、Webブラウジング、コード実行、さらにはエージェント同士の協調動作——自律的に行動を連鎖させるAIエージェントは、生産性を劇的に引き上げる一方で、制御設計を誤れば深刻なリスクとなる。本コラムでは、2026年4月現在の最新フレームワーク動向を踏まえ、企業がAIエージェントを安全に運用するためのガードレール設計の実践知を整理する。

エージェント導入の加速と"制御の空白"

OpenAIのOperator、GoogleのProject Mariner、AnthropicのClaude Agent、そしてMicrosoftのCopilot Studioにおけるエージェント機能の拡充——2025年から2026年にかけて、主要プレイヤーが相次いでエージェントプラットフォームを商用展開した。LangGraphやCrewAI、AutoGenといったオープンソースのオーケストレーションフレームワークも成熟し、マルチエージェント構成を比較的容易に構築できる時代になった。

しかし、導入の速度に対して「制御設計」の標準化は明らかに遅れていた。初期導入企業の多くは、プロンプトによる行動制約（いわゆる"システムプロンプトガードレール"）に依存し、構造的な権限管理や承認フローを組み込まないまま本番運用に入っていた。その結果、エージェントが想定外のAPI呼び出しを連鎖させたり、機密データを外部サービスに送信したりするケースが報告されるようになったのである。

最新フレームワークに見る制御機構の体系化

2026年現在、主要フレームワークが実装する制御機構は、おおむね以下の4層に体系化されつつある。

第1層：アクション許可リスト（Allowlist/Denylist）。 エージェントが呼び出せるツールやAPIを明示的にホワイトリスト/ブラックリストで管理する。LangGraph v0.4では「Tool Policy」としてYAMLベースの宣言的定義が導入され、実行時にポリシーエンジンがツール呼び出しをフィルタリングする。

第2層：Human-in-the-Loop（HITL）承認フロー。 一定のリスクスコアを超える操作、または金額・データ範囲が閾値を超えるアクションに対して、人間の明示的承認を挟む仕組みだ。AnthropicのAgent SDKが2026年初頭に導入した「Approval Gate」は、操作の種類ごとに承認レベルを3段階で設定でき、Slackやメールへの通知統合もネイティブサポートしている。

第3層：実行サンドボックスとリソース制約。 コード実行やファイルシステム操作を隔離環境で行い、ネットワークアクセス・メモリ・CPU時間に上限を設ける。MicrosoftのAutoGen v0.5が採用したコンテナベースのサンドボックスは、エージェントごとにネットワークポリシーを分離し、横方向の権限昇格を防ぐ設計が注目を集めている。

第4層：監査ログとリアルタイム監視。 エージェントの全アクション（思考ステップ、ツール呼び出し、入出力データ）を構造化ログとして記録し、異常検知モデルでリアルタイム監視する。LangSmithやArize Phoenixといったオブザーバビリティ基盤が、エージェント専用のトレーシング機能を強化しており、「なぜその行動を選択したのか」を事後検証可能にしている。

企業導入で押さえるべき5つの実践原則

フレームワークの機能を活かすためには、組織的な設計原則が不可欠だ。先行導入企業の知見を踏まえ、以下の5原則を提示したい。

1. 最小権限の原則を徹底する。 エージェントに与える権限は、タスク遂行に必要な最小限に絞る。「とりあえず全ツールを有効化」は最大のアンチパターンだ。

2. 段階的自律性の拡大（Progressive Autonomy）。 最初はすべてのアクションにHITL承認を要求し、運用実績とリスク評価に基づいて段階的に自律範囲を広げる。NISTが2026年3月に公開した「AI Agent Risk Management Framework（Draft）」でも、このアプローチが推奨されている。

3. フェイルセーフとしてのキルスイッチ。 エージェントの動作を即座に停止できる機構を必ず用意する。タイムアウト、コスト上限、ステップ数上限の3つは最低限設定すべきだ。

4. マルチエージェント構成における信頼境界の明確化。 エージェント間の通信にも認証・認可を適用し、あるエージェントが侵害された場合の影響範囲（Blast Radius）を限定する。

5. レッドチーミングの定期実施。 プロンプトインジェクション、間接的プロンプトインジェクション（外部Webページやドキュメント経由）、ツール呼び出しの悪用シナリオを定期的にテストする。OWASP が2026年版として更新した「Top 10 for LLM & Generative AI」にはエージェント特有の攻撃ベクトルが新たに追加されており、チェックリストとして有用だ。

ガードレールは"制約"ではなく"信頼の基盤"

エージェントにガードレールを設けることを「AIの能力を制限すること」と捉える向きもある。しかし実態は逆だ。適切な制御機構があるからこそ、経営層は安心してエージェントに高い自律性を委譲でき、結果としてAIが発揮できる価値の総量は大きくなる。ガードレールは制約ではなく、信頼の基盤である。

2026年はAIエージェントが「実験」から「本番運用」に移行する決定的な年になるだろう。その転換点において、制御設計の巧拙が企業のAI活用の成否を分ける。フレームワークの進化を追いつつ、自社のリスクプロファイルに合わせたガードレールを設計・運用し続けること——それが、AIエージェント時代を安全に生き抜くエンジニアリングの核心である。

ローカルLLM 2026年春——Llama 4とオンデバイス推論の現在地

「このレスポンス、本当にローカルで動いているのか？」——2026年春、手元のノートPCやスマートフォンでLLMを走らせたとき、多くのエンジニアが抱く率直な感想だろう。クラウドへのリクエストなし、ネットワーク遅延ゼロ。ローカルLLMは、もはや"趣味の実験"ではなく実務の選択肢になりつつある。

Llama 4ファミリーが変えた地殻

2025年4月にMetaが公開したLlama 4ファミリーは、Scout（109Bパラメータ/16エキスパート）とMaverick（400Bパラメータ/128エキスパート）のMixture-of-Experts（MoE）アーキテクチャを採用し、推論時にアクティブ化されるパラメータ数を大幅に抑えた。続く2026年初頭のBehemothの段階的公開により、性能面ではGPT-4oクラスに迫るオープンウェイトモデルが手に入る時代となった。特にScoutは128Kトークンという長大なコンテキスト長を持ちながらアクティブパラメータが約17Bに収まるため、ローカル推論との相性がきわめて良い。

量子化技術の"実用ライン"

ローカル実行を支えるのが量子化技術の進化だ。2025年後半に登場したGPTQベースの改良手法やllama.cppのimatrix対応Q4_K_Mプロファイルにより、4bit量子化でもベンチマーク劣化を3〜5%以内に抑えられるケースが増えた。さらに2026年に入りQLoRAとAWQを組み合わせたハイブリッド量子化が注目を集めており、ScoutクラスのMoEモデルを16GB VRAM帯のGPU1枚で毎秒20トークン超の速度で走らせる報告が相次いでいる。

オンデバイスチップの競争激化

ハードウェア側の追い風も大きい。Appleは2025年秋のM4 Ultra以降、Neural Engineのスループットを世代比で約40%向上させ、iPhone 17 ProのA19 Bionicではオンデバイスで7B級モデルのリアルタイム推論を公式にサポートした。一方QualcommはSnapdragon X Elite/8 Elite世代のHexagon NPUを強化し、Windows on Arm機でのローカルLLMユースケースをOEM各社と共同で推進中だ。2026年春時点で、ノートPC・スマートフォンの双方に「NPUで数十億パラメータを動かせる」土壌が整ったと言える。

エンジニアが今試すべきこと

手始めに推奨したいのは、**Llama 4 ScoutのQ4_K_M量子化モデル**をllama.cppまたはollamaで動かす構成だ。16GB以上のメモリを搭載したマシンであれば、コーディング支援や社内ドキュメントRAGの検証が即日始められる。プライバシー制約の厳しい業務、オフライン環境での推論、エッジデバイスへの組み込みなど、クラウドAPIでは対応しにくい領域こそローカルLLMの主戦場だ。

2026年春、「どこで推論するか」は性能だけでなくコスト・セキュリティ・レイテンシの総合判断になった。手元のGPUとNPUの実力を、まずは自分の手で確かめてみてほしい。